WebXR: Oral Exam Study Guide

I.BA_WEBXR.H2601 · Intro to WebXR · HSLU · Compiled 10.11.2026 for the oral exam on Wed 11.11.2026 (block 09.A, 12:50)

How to use this guide. Every chapter is marked with where its content comes from:

From lecture summarised from the teacher's course PDFs (SW01–SW06)

General knowledge listed on the course calendar, but there was no PDF for it in the repo. These notes are standard background knowledge; the teacher's wording may differ.

Watch out: The format of the oral exam was explained only in class (calendar SW05: "Discuss Oral Exam"). None of the PDFs describe it. Check Ilias or ask a classmate who had the exam on 04.11.

Contents

1. Course calendar & what can be asked
2. What is WebXR?
3. Timeline of the WebXR standard
4. Status today, browsers, W3C process
5. WebAR workarounds (8th Wall, MindAR, AR.js)
6. AR tracking types & good target images
7. Web dev setup: HTML, Git, Node, Vite, HTTPS
8. Three.js & A-Frame + "who owns the ground"
9. A-Frame VR scene with examples
10. MindAR image-tracking scene with examples
11. 3D models for the web (glTF, Khronos, WebGL/WebGPU)
12. Optimizing in Blender & compressing
13. Models in the scene, measuring, animation
14. Scene lighting
15. Git & GitLab
16. Usability features (all 8 areas + code)
17. How hosting works & how to host
18. Basic security features
19. Your project: Endless Line
20. Practice questions with model answers
21. Glossary & numbers to memorise

1 · Course calendar & what can be asked From lecture

Room I.S41_309, Wednesdays 12:50–15:10. The oral exams are on **04.11 (08.B)** and **11.11 (09.A)**. Everything taught up to SW08 can come up.

Week	Date	Topics	Chapter here
SW01	16.09	About this course · What is WebXR? · Timeline of WebXR · WebAR workarounds · Testing web-based VR & AR	2–5
SW02	23.09	Basic web dev setup · What are A-Frame & Three.js · Basic A-Frame VR scene · Exercise *Homework: finish the VR scene (HTTPS, runs on headset & phone)*	7–9
SW03	30.09	Digesting the A-Frame scene · Exercise: changes using only the A-Frame docs · Free experimentation	9
SW04	07.10	Types of AR tracking possible on the web · What makes a good target image · Exercise: basic MindAR scene	6, 10
SW05	14.10	3D models on the web · Optimizing & compressing · Discuss project scope · **Discuss oral exam** · Project proposal	11–13
SW06	21.10	(Scene lighting) · GitLab · Basic usability features · *Hand in proposal*	14–16
SW07	28.10	Project proposal feedback · start working	19
SW08	04.11	How hosting works · How we will host it · Basic security features · **Oral exams**	17–18
SW09	11.11	**Oral exams** · work on project · teacher support	
SW10–13	18.11–09.12	Work on project · Hosting it, last feedback, troubleshooting (25.11)	
SW14	16.12	Presenting the finished hosted project	19

Typical oral-exam question patterns

- **"Explain…"**: What is WebXR? What is SLAM? What does Draco do?
- **"Why…"**: Why HTTPS? Why a workaround instead of WebXR on iPhone? Why GLB?
- **"Compare…"**: A-Frame vs. Three.js, Three.js vs. Babylon.js, WebGL vs. WebGPU, WebP vs. KTX2, marker vs. image tracking.
- **"Apply it to your project…"**: which tracking, which tools, which usability features, what the risks are.
- **"Who / when"**: W3C, Khronos, Mozilla, Niantic, Supermedium, Mr.doob, Cloudflare/VoidZero, plus key years.

Remember: For every concept, prepare a one-sentence definition, one example, and one link to your own project. That structure works for almost any oral question.

2 · What is WebXR? From lecture · SW01

The **WebXR Device API** works like a **translator between a website and VR/AR hardware**: headsets, motion sensors (also built into smartphones) and hand controllers. A normal website can "talk" to your mouse and keyboard; WebXR lets it "talk" to a headset too. The browser can then display a full 3D world (**VR**) or put digital objects on top of the real world (**AR**).

- It's developed by the **W3C** (World Wide Web Consortium), the international organisation that maintains the web's core standards (HTML, CSS, …), inside the **Immersive Web Working Group**.
- It's still a work in progress, but several browsers already implement it.
- **Main benefit:** before WebXR, almost every VR/AR experience needed its own installed app for one platform. With WebXR you **build once and run in the browser: no install, no app store, no platform lock-in**.
- The W3C's goal: websites should be able to do **everything a native AR/VR app can do**, not offer a weaker substitute.

> W3C, on the first public draft (2019): "*The ability to interact directly with immersive hardware is critical to ensuring that the web is well equipped to operate as a first-class citizen in this environment. … The WebXR Device API provides the interfaces necessary to enable developers to build compelling, comfortable, and safe immersive applications on the web across a wide variety of hardware form factors.*"

Key terms

Term	Meaning	Example
VR	Virtual Reality: you are fully inside a computer-generated world	A-Frame scene on a Quest headset
AR	Augmented Reality: digital objects on top of the real camera view	Model standing on a postcard (MindAR)
XR	"Extended reality": umbrella term for VR + AR (+ MR)	WebXR covers both
3DoF / 6DoF	Degrees of freedom: 3 = rotation only; 6 = rotation + position (moving through space)	SLAM gives 6DoF

Example: what the raw WebXR API looks like General knowledge

A-Frame does this for you (the "Enter VR/AR" button), but knowing it helps you explain what WebXR is:

```
// 1. Feature detection: does this browser have WebXR, and does it support AR?
if (navigator.xr && await navigator.xr.isSessionSupported('immersive-ar')) {
  // 2. A session must be started from a user gesture (button click)
  startButton.onclick = async () => {
    const session = await navigator.xr.requestSession('immersive-ar', {
      requiredFeatures: ['hit-test'],          // WebXR Hit Test Module
      optionalFeatures: ['dom-overlay', 'anchors'],
      domOverlay: { root: document.getElementById('ui') } // HTML UI on top
    });
    // 3. The renderer (Three.js/A-Frame) then draws a frame for every
    //    session.requestAnimationFrame(...) using the headset/phone pose.
  };
} else {
  showFallback(); // e.g. Safari on iPhone -> no WebXR AR
}
```

Testing web-based VR and AR

- Desktop browser: fast for layout and logic; you see the 3D scene but not the headset or camera.

- Real phone or headset on the **same Wi-Fi**, via the **HTTPS** address Vite gives you (see chapter 7).
- On-device debugging: `<a-scene stats>` for fps and draw calls, and **Eruda** as a developer console on the phone (chapter 13), or USB remote debugging.
- General The Immersive Web Emulator browser extension can simulate a headset on the desktop.

3 · Timeline of the WebXR standard From lecture · SW01

Date	Event
Undated (before 2018)	**WebVR**, created by **Mozilla**. It was the catalyst: it showed browser XR was possible. It was **never ratified** as a W3C standard, was on by default in only a few browsers, and supported few devices. Now **deprecated** in favour of WebXR.
26 April 2018	Launch of **Mozilla Hubs**, a browser-based social VR experiment and one of the best-known web VR examples. Mozilla **ended support in 2024**; the **Hubs Foundation** maintains the code now.
24 Sept 2018	Start of the W3C **Immersive Web Working Group**, whose goal is to develop the WebXR Device API.
5 Feb 2019	**First Public Working Draft** of the WebXR Device API.
10 Dec 2019	**Chrome 79** (Blink engine): **the first browser** to ship WebXR.
28 Feb 2020	**Brave** (Chromium-based) turns WebXR on by default (before that it was a disabled feature).
14 April 2020	**Microsoft Edge 81** (Chromium-based) reaches the stable channel; full WebXR is documented from this version on.
2020–2024	Modules ship: **Hit Test** (Chrome/Edge 81, 2020), **Anchors** (85, 2020), **Depth Sensing** (90, 2021), **Hand Input** (Chrome 131, 2024).
Sept 2026	The WebXR Device API is a **Candidate Recommendation Draft** (draft dated 9 June 2026).

Remember: Use this story for "explain the history": Mozilla's WebVR proved the idea → the W3C formed a working group (2018) → first draft (2019) → Chrome 79 first (Dec 2019) → Brave and Edge (2020) → today a Candidate Recommendation Draft, fully supported only by Blink-based browsers.

4 · Status today, browsers & the W3C process From lecture · SW01

W3C maturity stages

The W3C has several maturity stages. "**Candidate Recommendation**" sits just below the highest stage, "**Recommendation**". "**Draft**" is a sub-stage inside Candidate Recommendation. WebXR is there now (Sept 2026), and so is **WebGPU** (chapter 11).

```
Working Draft  →  Candidate Recommendation (Snapshot / DRAFT ← WebXR is here)  →  Proposed Rec.  →  RECOMMENDATION
```

Browser support (Table 1 of the lecture)

Browser	Engine	WebXR status	Europe users (Aug 2026)
Chrome	Blink (Chromium)	Full	62.66 %
Edge	Blink (Chromium)	Full	6.66 %
Brave	Blink (Chromium)	Full	counted as Chrome*
Safari	WebKit	**Partial**: VR only, Vision Pro only	17.8 %
Firefox	Gecko	**None**	4.61 %
Meta Quest Browser	Blink (Chromium)	Full	–
PICO Browser	Blink (Chromium)	Full	–

* Brave sends the same User-Agent header as Chromium, so StatCounter can't tell it apart from Chrome. Headset browsers aren't tracked by mainstream market-share tools.

Watch out: Consequence: **AR through WebXR doesn't work on any iPhone/iPad** (Safari, and every iOS browser uses WebKit). Around 18 % of European users are on Safari. That's why the WebAR workarounds exist (chapter 5).

Outlook: when does it become a "Recommendation"?

- No date can be predicted. The working group charter calls WebXR a **"living specification"** with **yearly updates** ("Expected W3C Recommendation: yearly basis").
- **Decisive criterion:** at least **two independent browser engines** with a complete, verifiably interoperable implementation.
- So far only **Blink** has one. **Firefox/Gecko** doesn't implement WebXR, and **Mozilla is no longer a member** of the working group. **Apple/WebKit** co-chairs the working group, but that doesn't guarantee a full implementation.

What would change with Recommendation status?

1. **Patent protection / royalty-free use:** "A W3C Recommendation has commitments from Working Group members to royalty-free licensing for implementations." Only at that stage do members make a **legally binding commitment** to license their patents royalty-free. Before that, this safeguard doesn't formally exist.
2. "W3C recommends the **wide deployment** of a Recommendation as a standard for the Web." Whether this would actually make Mozilla or Apple implement it can't be proven from the sources.

5 · WebAR workarounds From lecture · SW01 + SW04

Problem: the WebXR AR module doesn't work in Safari (desktop, iOS, iPadOS).
How the workarounds solve it: they **don't use the WebXR Device API at all**. They run their **own camera access, computer vision and rendering in JavaScript/WebGL**, which works in all browsers. Goal: **web AR on every browser with one build.**

8th Wall
Owned and maintained by **Niantic Spatial, Inc.** (a company).
Since **March 2026** the engine is open source (**MIT**), but the **SLAM module stays closed-source** (binary only).
Can do: **SLAM/world tracking**, face effects, image targets (also **curved**), **sky effects**.
How: its own SLAM engine maps the environment from the camera.

MindAR
Open source, **MIT**. Maintained by **one individual developer, "hiukim"** on GitHub.
Can do: **image tracking** and **face tracking**.
How: image tracking uses **TensorFlow.js purely as a GPU compute engine** (not machine learning in the AI sense); face tracking uses a face-landmark/mesh model.

AR.js
Open source, MIT (+ LGPLv3 for artoolkit5-js). Maintained by the **AR.js org**, a GitHub organisation rather than one person or a company.
Can do: **marker tracking**, **location-based (GPS) AR**, **image tracking (NFT)**.
How: marker recognition, GPS placement and image tracking in plain JavaScript using WebGL and the device's camera/GPS APIs.
Note: one SW04 slide says "AR.js owned by Niantic Spatial". SW01 and its sources say the AR.js org, so go with that.

Comparison table (teacher's own research; "confirm yourself, the landscape changes")

Capability	WebXR (native)	8th Wall	MindAR	AR.js
Maintainer	W3C (multi-vendor)	Corporation (Niantic Spatial)	Individual developer	Community org
License	Open web standard	Mixed: MIT engine + closed SLAM	MIT	MIT (+LGPLv3)
World tracking (SLAM)	Not part of the spec	Yes (closed binary)	No	No
Hit test (surface placement)	Yes	Yes	No	No
Marker tracking	No	No	No	**Yes**
Image tracking	No	Yes	**Yes**	Yes (NFT)
Face tracking	No	Yes	**Yes**	No
Location-based (GPS)	No	No	No	**Yes**
Depth sensing / occlusion	Yes	Limited (mesh occlusion + sky segm.)	No	No
Anchors (persistent placement)	Yes	No	No	No
Hand input	Yes	No	No	No

Remember: Image, marker and face tracking are **not** in any shipping WebXR module. The "WebXR Marker Tracking Module" is only a Draft Community Group Report (July 2021), and face tracking is confirmed absent on WebXR's GitHub (issue #1150). Native WebXR is stronger at hit test, anchors, depth and hands.

How to choose (example reasoning)

- Furniture placed on the floor, must work on iPhone → **8th Wall** (SLAM + hit test). With Android/Quest only → native WebXR hit test.
- Postcard or ticket that comes alive → **MindAR** image tracking (free, MIT, Safari + Chrome). *This is your project.*
- Sunglasses or a mask try-on → **MindAR** or 8th Wall face tracking.
- City tour with points of interest → **AR.js** location-based.
- Many robust IDs, e.g. in a classroom or factory → **AR.js** markers.

6 · AR tracking types & good target images From lecture · SW04

World tracking: SLAM (8th Wall)

SLAM = Simultaneous Localisation And Mapping. The device works out where it is (localisation) while building a map of the room (mapping), both at once.

- Uses the **camera feed**: tracks **feature points** and builds a **point cloud**.
- Uses the **gyroscope** (rotation) and **accelerometer** (gravity and acceleration direction) to measure the user's movement in **6DoF**.
- Extras: **plane/surface detection** (find the floor), **hit test** (tap on the 2D screen, place in 3D space), **lighting estimation** (brightness and colour of the ambient light), **world-scale content** (real-world size).
- Example: **IKEA** furniture placement (app-based, but 8th Wall can do the same on the web, and IKEA's online store has a web version for some items).

Sky effects (8th Wall)

- **Segmentation:** a neural network labels every camera pixel as "sky" or "not sky".
- **Mask:** a black-and-white mask decides where your texture or objects are drawn (e.g. replace the sky with an aurora).

Face tracking (8th Wall, MindAR)

A machine-learning model finds a face in the camera feed and predicts a **3D mesh of landmarks** on it. 3D content (glasses, hats) is attached to that mesh. Example: the MindAR live demo.

Location-based (AR.js)

- User position from **GPS** (longitude/latitude); viewing direction from the **compass + orientation sensors** (gyroscope, accelerometer, magnetometer). Objects can be **scaled by distance** to the user.
- **Issues:** GPS is only **3–10 m accurate**; the compass is unreliable **inside buildings or near metal**; there's **no SLAM** built in. So it only works in **big, open, forgiving outdoor spaces**.
- **Fun fact:** Pokémon Go combined GPS with local SLAM. Its players became the data source for Niantic's visual map (**VPS, Visual Positioning System**). Niantic says VPS + SLAM can place content with **centimetre accuracy**.

Marker tracking (AR.js)

- Markers are designed to be easy for computers to read: simple **black-and-white squares**, **thick borders** for recognition, and an image inside the border to tell markers apart.
- 3D content is placed relative to the marker.
- **Use markers when robustness, speed or many IDs matter more than looks.**

Image tracking (MindAR, AR.js, 8th Wall)

- Recognises a **known 2D image** in the camera feed and calculates its **position and rotation relative to the camera**. 3D content is placed relative to the image.
- **Curved** image targets (cans, bottles) are an 8th Wall-only feature.
- Marker vs. image: a marker is built for the computer (robust, ugly); an image target can be any good-looking picture but needs enough detail.

What makes a good target image?

Good ✓	Bad ✗
Varied and abundant detail	Repetitive patterns (tiles, checkerboards)
High contrast (think black & white)	Too much blank space, low contrast, soft gradients
Sharp corners & lines	Blurry, rounded, smooth shapes
Non-reflective (matte) surface	Glossy or reflective print, screens with glare

Why? The tracker looks for **feature points** (corners, edges with high contrast). Repeated patterns make different points look the same, so the tracker can't tell where it is. Blank areas have no points at all, and reflections change the image as you move.

Your project example: the A6 ticket card is designed as a target: high contrast, sharp edges, **asymmetric** details, no SBB logo.

7 · Web dev setup: HTML, Git, Node, Vite, HTTPS From lecture · SW02

Installs

Tool	Purpose
VS Code	Code editor: a text editor built for code, with a built-in terminal
Git	Saves a history of every change so you can go back to any earlier version
GitHub account	Online home for Git projects: backup + sharing (also with the teacher)
GitHub Desktop	Visual Git tool the teacher uses (free, open source, by GitHub)
Node.js (LTS)	Runs JavaScript tools on your computer; ships with **npm** (package manager). Check with `node -v` and `npm -v`

HTML basics: hello world

```
<!DOCTYPE html>
<html>
  <head>                          <!-- information ABOUT the page, not shown -->
    <meta charset="UTF-8">        <!-- encoding: ä, é, emoji display correctly -->
    <title>Hello World</title>    <!-- browser tab, bookmarks, search results -->
  </head>
  <body>                          <!-- everything the visitor sees -->
    <h1>Hello World</h1>
    <p>This is my first paragraph.</p>   <!-- the SW02 exercise -->
  </body>
</html>
```

- **HTML** describes **structure and meaning** with nested tags. It doesn't control looks (that's **CSS**), and it can't make things move or react (that's **JavaScript**).
- `index.html` is the file a browser **looks for first** when it opens a folder as a website.

Git vocabulary

- **Clone**: download your own copy of the repository, kept in sync with GitHub/GitLab.
- **Commit**: a saved snapshot of the project with a note about what changed. **Push**: upload commits to the server.
- A **clean commit history** helps your future self understand the project.

Vite: local dev server

Why Vite? It's a local development server, and it makes it **easy to add HTTPS**, which WebXR needs for testing on a headset or phone.

```
# 1. create the project (pick "Vanilla" / "JavaScript")
npm create vite@latest
# 2. delete Vite's demo content -> commit & push
# 3. add HTTPS support (one-time)
npm install --save-dev @vitejs/plugin-basic-ssl
# 4. start the dev server (run it in the folder that contains index.html!)
npm run dev        # stop with Ctrl + C
```

```
// vite.config.js  (in the project root)
import { defineConfig } from 'vite'
import basicSsl from '@vitejs/plugin-basic-ssl'

export default defineConfig({
  plugins: [basicSsl()],     // self-signed HTTPS certificate
  server: { host: true },    // reachable from other devices on the same Wi-Fi
})
```

Why HTTPS?

- **WebXR and camera access only work in a "secure context"**, so the page must come over **HTTPS**, not plain HTTP.
- On the desktop, `localhost` counts as secure. But the phone opens your PC's IP address, e.g. `https://192.168.1.23:5173`, so it needs HTTPS.
- The first time, the browser warns "**Your connection is not private**". That's expected: the certificate is **self-signed**, not issued by a trusted authority. Click "Advanced" → proceed.
- Troubleshooting: if the scene doesn't appear on the headset, restart `npm run dev`.

8 · Three.js & A-Frame, and "who owns the ground we build on?"

From lecture · SW02

Three.js (threejs.org)
A **JavaScript library** that makes 3D graphics in the browser practical. It wraps **WebGL** (the browser's low-level 3D API) into simple building blocks: **scene, camera, lights, 3D objects**, all in plain JavaScript.
Created by **Ricardo Cabello ("Mr.doob")**, first released on GitHub in **April 2010**. Community-driven, open source, not owned by any single company.
A large toolbox that saves you from writing raw WebGL.

A-Frame (aframe.io)
A free, open-source framework **built on top of Three.js**. You build VR/WebXR scenes with **declarative HTML-like tags** (`<a-scene>`, `<a-box>`, `<a-sky>`) instead of writing JavaScript.
Started in **Mozilla's VR team**, first released **December 2015**. Now maintained by its co-creators at **Supermedium**, a small San Francisco startup founded by three former Mozilla WebVR team members.
Everything you write in A-Frame becomes Three.js underneath.

Same box, two ways (example for "compare A-Frame and Three.js")

```
<!-- A-Frame: declarative -->
<a-scene>
  <a-box position="0 1 -3"
         color="#4CC3D9"></a-box>
</a-scene>
<!-- camera, renderer, render loop,
     "Enter VR" button: automatic -->
```

```
// Three.js: imperative
const scene = new THREE.Scene();
const camera = new THREE.PerspectiveCamera(
  75, innerWidth/innerHeight, 0.1, 100);
const renderer = new THREE.WebGLRenderer();
renderer.setSize(innerWidth, innerHeight);
document.body.appendChild(renderer.domElement);
const box = new THREE.Mesh(
  new THREE.BoxGeometry(),
  new
THREE.MeshStandardMaterial({color:'#4CC3D9'}));
box.position.set(0, 1, -3);
scene.add(box, new THREE.AmbientLight());
renderer.setAnimationLoop(() =>
  renderer.render(scene, camera));
```

The stack from bottom to top: **GPU → WebGL (browser API) → Three.js (library) → A-Frame (framework, HTML tags) → your scene**. On top of that, **WebXR** supplies the headset/phone pose, and **MindAR** supplies the camera image and the tracked image position.

Who owns the ground we're building on?

Tool	Ownership & risk
Vite	Created by **Evan You** (also creator of Vue.js), who founded **VoidZero** around it in 2024. In **June 2026 Cloudflare acquired VoidZero**. Cloudflare says Vite **stays MIT-licensed, open source and "vendor-agnostic"** (you're not locked into Cloudflare), is still developed in the open by the existing team, and Cloudflare funds a **$1 million** fund for outside contributors.
Three.js	MIT, broad community (100k+ GitHub stars), **not dependent on Cabello personally**. Low risk.
A-Frame	Also MIT (anyone could fork it), but active maintenance is **concentrated in one small company** (Supermedium). That's closer to MindAR's situation than to Three.js's.
MindAR	MIT, but **one single developer**. Highest "bus factor" risk.

Remember: An acquisition doesn't automatically change how you can depend on a tool. What matters is what happens to the **license and governance**. The **MIT license is your safety net**: if a maintainer disappears, anyone is legally free to take the code and keep going.

9 · A-Frame VR scene, with examples From lecture · SW02/SW03

The basic scene (from A-Frame's introduction page)

```
<html>
  <head>
    <script src="https://aframe.io/releases/1.8.0/aframe.min.js"></script>
  </head>
  <body>
    <a-scene>
      <a-box      position="-1 0.5 -3" rotation="0 45 0" color="#4CC3D9"></a-box>
      <a-sphere   position="0 1.25 -5" radius="1.25"     color="#EF2D5E"></a-sphere>
      <a-cylinder position="1 0.75 -3" radius="0.5" height="1.5" color="#FFC65D"></a-cylinder>
      <a-plane    position="0 0 -4" rotation="-90 0 0" width="4" height="4" color="#7BC8A4"></a-plane>
      <a-sky color="#ECECEC"></a-sky>
    </a-scene>
  </body>
</html>
```

Remember: A-Frame handles the 3D setup, the **VR entry button** and the **default camera controls** automatically.

How to read it

- **Entity-Component-System (ECS)** General : every tag is an **entity**; attributes like `position`, `color`, `animation` are **components** that add data or behaviour. `<a-box>` is a shortcut ("primitive") for `<a-entity geometry="primitive: box" material="color: …">`.
- **Units are metres**; `position="x y z"` : x = right, y = up, **−z = forward** (into the screen). That's why objects sit at z = −3.
- `rotation` is in **degrees**: `rotation="-90 0 0"` lays the plane flat as a floor.

SW02/SW03 exercise solutions (examples)

```
<!-- 1. change the colour of an object -->
<a-box position="-1 0.5 -3" rotation="0 45 0" color="#FF8800"></a-box>

<!-- 2. add another object (only A-Frame geometries) -->
<a-torus position="0 2 -4" radius="0.5" radius-tubular="0.05" color="#8E44AD"></a-torus>
<a-cone  position="2 0.5 -4" radius-bottom="0.5" height="1" color="#2ECC71"></a-cone>

<!-- 3. add an animation -->
<a-box position="-1 0.5 -3" color="#4CC3D9"
       animation="property: rotation; to: 0 360 0; loop: true; dur: 4000; easing: linear"></a-box>

<a-sphere position="0 1.25 -5" radius="1.25" color="#EF2D5E"
       animation="property: position; to: 0 2 -5; dir: alternate; loop: true; dur: 1500"></a-sphere>

<!-- 4. experiment: react to a click (desktop: cursor follows the mouse) -->
<a-scene cursor="rayOrigin: mouse" raycaster="objects: .clickable">
  <a-box class="clickable" position="0 1 -3"
         animation__click="property: scale; to: 1.5 1.5 1.5; startEvents: click; dur: 300"></a-box>
</a-scene>
```

Exercise rule: **use only the A-Frame documentation** (no AI, no imported assets). Commit and push after every step.

Writing your own component (JavaScript)

```
<script>
AFRAME.registerComponent('spin', {
  schema: { speed: { type: 'number', default: 90 } },     // degrees per second
  tick: function (time, timeDelta) {                      // runs every frame
    this.el.object3D.rotation.y += THREE.MathUtils.degToRad(this.data.speed) * timeDelta / 1000;
  }
});
</script>
<a-box spin="speed: 45" position="0 1 -3"></a-box>
```

Note `timeDelta`: movement is scaled by time, not by frame count, so the speed is the same at 30 or 90 fps.

10 · MindAR image-tracking scene, with examples From lecture · SW04/SW05

Workflow

1. Choose a good target image (chapter 6).
2. Compile it to a `targets.mind` file with the MindAR image-target compiler (web tool).
3. Put `targets.mind` in Vite's `public/` folder.
4. Build the scene, run `npm run dev`, and open the HTTPS address on the phone. Point it at the printed image and the box appears.

Basic MindAR scene

```
<head>
  <script src="https://aframe.io/releases/1.5.0/aframe.min.js"></script>
  <script src="https://cdn.jsdelivr.net/npm/mind-ar@1.2.5/dist/mindar-image-aframe.prod.js"></script>
</head>
<body>
  <a-scene mindar-image="imageTargetSrc: /targets.mind;"
           color-space="sRGB" renderer="colorManagement: true"
           vr-mode-ui="enabled: false" device-orientation-permission-ui="enabled: false">

    <a-camera position="0 0 0" look-controls="enabled: false"></a-camera>

    <!-- everything inside the target is positioned RELATIVE TO THE IMAGE -->
    <a-entity mindar-image-target="targetIndex: 0">
      <a-box position="0 0 0.25" scale="0.5 0.5 0.5" color="#4CC3D9"></a-box>
    </a-entity>
  </a-scene>
</body>
```

Use the exact A-Frame/MindAR versions from the class example. MindAR builds are tied to specific A-Frame versions.

The image-target coordinate system (very likely question)

- The **centre of the image is the origin** (0 0 0).
- **1 unit = the width of the image** (not 1 metre!).
- The **z axis points out of the image toward you**.
- Example: the box is scaled to **0.5** (half the image width) and moved **0.25 forward on z** (half its own size), so it **rests exactly on the image** instead of sinking into it.

Events: the basis for usability hints

```
<script>
const target = document.querySelector('[mindar-image-target]');
const hint   = document.getElementById('hint');            // "Point your camera at the ticket"

target.addEventListener('targetFound', () => { hint.hidden = true;  });
target.addEventListener('targetLost',  () => { hint.hidden = false;
                                               hint.textContent = 'Ticket lost, point at it again'; });

const scene = document.querySelector('a-scene');
scene.addEventListener('arReady', () => console.log('camera running'));
scene.addEventListener('arError', () => showError('We could not start the camera.'));
</script>
```

Restyling MindAR's default UI (SW06 "consistent styling"): MindAR has built-in loading, scanning and error overlays. Turn them off or point them to your own elements, e.g. `mindar-image="imageTargetSrc: /targets.mind; uiScanning: #myScan; uiLoading: no"`.

11 · 3D models for the web: glTF, Khronos, WebGL/WebGPU

From lecture · SW05

File format: glTF / GLB

- **glTF (GL Transmission Format)** is **the standard file format for 3D on the web**. Khronos calls it **"the JPEG of 3D"**.
- Two variants: **.gltf** is a **JSON** file that often references separate texture and binary files; **.glb** is **one self-contained binary file** that bundles everything. **The course uses .glb.**
- Made and maintained by the **Khronos Group**. **1.0 in 2015, 2.0 in 2017**. In **2022**, glTF 2.0 became the international standard **ISO/IEC 12113:2022**.
- glTF is always **+Y up** and **measured in metres**. In a MindAR scene, 1 unit = the image width, so you scale the model to fit.
- General Why not FBX or OBJ? FBX is a proprietary Autodesk format, large and slow to parse on the web. That's why the teacher's `CFS-RoboDogRocket.fbx` (~14 MB) had to be converted. OBJ has no animations and no PBR materials.

The Khronos Group

- An industry **consortium** that writes the open standards your graphics card, browser and headset use to talk to each other: **OpenGL and Vulkan** (graphics cards), **WebGL** (browsers), **OpenXR** (XR devices), **glTF**.
- Founded in **2000**; **not-for-profit, funded by its members**. It works like the W3C, but focuses on **graphics, 3D and XR** instead of the web in general. Goal: open, **royalty-free** standards.

Remember: Khronos writes the standards; other companies build the software and hardware that implement them. (Compare: the W3C writes WebXR and WebGPU, and browsers implement them.)

Around the corner: interactive GLBs (KHR_interactivity)

A glTF extension that embeds **behaviour graphs** (networks of nodes for events, logic and animation) **directly in the file**. A model can react without custom code for every platform, e.g. play an animation when the viewer comes close, or show/hide parts on input. Khronos submitted it for ratification on **16 July 2026**. **Babylon.js** and the **Khronos glTF Sample Viewer** can already play these graphs.

Good to know: Babylon.js vs. Three.js

	Three.js	Babylon.js
Origin	Mr.doob, 2010, community	Started at **Microsoft, 2013**, written in **TypeScript**
Type	**Lighter library** you assemble yourself	**Full engine** with built-in tools: scene inspector, physics, node material editor
Common	Both are free, open-source JS 3D engines; render with WebGL and WebGPU; support WebXR. They're **competitors**.	

Rendering in the browser: WebGL

- **WebGL is the browser's connection to the GPU**, the same way the WebXR Device API is the browser's connection to the headset. A-Frame/Three.js use WebGL to render everything in this course.
- **Web vs. native:** on the web there's an **extra layer**. A native app talks to the GPU almost directly through the OS; a website goes through the **browser's rendering pipeline and JavaScript engine** first. That used to

make browser graphics much slower, but modern GPUs and WebGL have made real-time 3D in the browser genuinely fast.

WebGL now, WebGPU tomorrow

- **WebGPU** is replacing WebGL. It's a newer graphics API built around **how modern GPUs actually work**. Like WebXR, it's developed by the **W3C** and is a **Candidate Recommendation Draft** (Sept 2026).
- **Why it's exciting:** more **computational budget** to "go bigger and busier". In simple scenes you won't see a difference, but **scenes with many objects or heavy GPU computing** run much faster.
- Demos: three.js WebGPU ocean, "False Earth" (Codrops).

12 · Optimizing in Blender & compressing From lecture · SW05

Pipeline: **1. optimize & export in Blender → 2. compress → 3. add to the scene → 4. measure performance.**

> **Remember:** Everything that makes a model heavy is decided **in Blender, before any compression**. Compression can only shrink what you hand it.

The four things that drive file size and performance

Trait	Why it costs	Numbers / example
Vertices	Every vertex stores position, normal and UV	≈ **32 bytes** per vertex → **100,000 vertices ≈ 3 MB** before compression
Textures	Usually the **biggest part** of the file and of GPU memory	**2048×2048 = 16 MB of GPU memory** uncompressed (2048 × 2048 px × 4 bytes RGBA)
Materials	**Every material on a mesh is its own draw call**	Same geometry with 20 materials renders slower than with 2
Animations	Keyframes for every bone add up; **shape keys (morph targets) are heavy**	Keep only the clips you need; keep the bone count low

Performance budget: think of a budget per scene and per model that you **spend** on these four traits. There's no exact target for each; you **distribute** the budget to get the look you want at the lowest cost. **Teacher's practical tip:** start by **merging materials and textures** (in number and size), then check where you are before spending time on vertex count and animations.

Merging textures, in a nutshell: create a **new UV map** that places all combinable texture areas onto **one texture**, then **bake** the albedo colour from the old UV map onto the new single texture. Then a **single material** applies that texture to the model.

Checklist before export

1. **Sensible size:** check the dimensions in the N-panel (Item tab). In MindAR, 1 unit = image width, so a model of about 1 m roughly fits the image width; fine-tune with `scale`.
2. **Apply transforms:** select all → **Ctrl+A** → **All Transforms** (scale 1, rotation 0).
3. **Set the origin** where the model should be anchored (e.g. the point that sits on the centre of the image): put the 3D cursor there → Object → Set Origin → **Origin to 3D Cursor**.
4. **Remove what nobody sees:** faces underneath and inside, hidden objects, cameras, lights, empties → File → Clean Up → **Purge Unused Data**.
5. **Reduce the mesh: Merge by Distance (M)**, a **Decimate modifier** for dense meshes (scans), and **bake fine detail into a normal map** instead of geometry. Target: **under 50,000 triangles per model**.
6. **Few materials:** join objects that share a material (**Ctrl+J**). Aim for **1–3 materials per model**.
7. **Principled BSDF only**: it's the **only shader that exports to glTF**. Procedural textures and custom node setups must be **baked into image textures** first.
8. **Textures sized to the need: 1024×1024** for small objects, **2048×2048 only for hero objects** seen up close. Use **powers of two** (512, 1024, 2048).
9. **Transparency:** with alpha, overlapping parts can render in the wrong order. Put a **Math node set to Round** between the texture's Alpha output and the BSDF's Alpha input (hard cut-out). Parts that need soft transparency (glass, fades) go in **their own object with Backface Culling on**.
10. **Animations:** delete unused Actions; keep the clip(s) you need.

11. **File name: lowercase, no spaces, no diacritics** (ä, é…), e.g. `lamp.glb`. Diacritics in particular cause trouble on web servers.

Export settings (File → Export → glTF 2.0)

- Format: **glTF Binary (.glb)** · Include: **Selected Objects only** · Transform: **+Y Up**
- Mesh: **Apply Modifiers**, UVs and Normals on · **Compression (Draco): OFF**, because you compress in the next step and want to **compare before and after**.
- Animation: on only if the model is animated (use the action filter).
- Then **check the file in a glTF viewer** (gltf-viewer.donmccurdy.com or the Khronos glTF-Validator) and **note the file size**.

Compression: file-size goals

There's no single correct number, but **Google's ARCore guidelines** give **ceilings, not targets**:

- Model size max **10 MB** ("bigger models may result in poor user experience")
- Triangles max **100,000**, with **30,000–50,000** as "an ideal range"
- Texture resolution max **2048×2048**

Course target: 3–5 MB per model, textures 1024 px (2048 max). If you're still above 5 MB, look at the biggest item first (`gltf-transform inspect`).

Mesh compression: Draco (by Google)

- Draco **doesn't delete detail**. It stores the vertex data in a much smaller form: positions, normals and UVs are **quantized** (stored with fewer bits instead of 32-bit floats) and then **entropy-coded**. The geometry typically gets **90–95 % smaller**.
- The device **unpacks it when the model loads, in a Web Worker**, so the page doesn't freeze.

> **Watch out: Draco makes the download smaller, not the rendering faster.** Frame rate depends on **triangles, draw calls (materials) and textures**.

Texture compression

Format	Download size	GPU memory	Notes
PNG / JPG	large	uncompressed	default from Blender
WebP	**noticeably smaller** at similar quality	**unchanged** (GPU still stores it uncompressed)	**Used in this course**, plus lower resolution
KTX2 (Basis Universal)	small	**about 4–8× smaller**: the only format that also shrinks GPU memory	needs an extra transcoder in the page + KTX-Software; "the next step for bigger projects"

Two things matter for textures: **resolution** (the smallest size that still looks good at the size it's actually shown) and **file format**.

Tool: gltf-transform

```
# one-time global install
npm install --global @gltf-transform/cli

# look at the model BEFORE: size, triangles, textures, draw calls
gltf-transform inspect robodog.glb

# Draco mesh compression + WebP textures + max 1024 px, in one step
gltf-transform optimize robodog.glb robodog-opt.glb --compress draco --texture-compress webp --texture-size
1024

# look AFTER and compare your numbers
gltf-transform inspect robodog-opt.glb
```

Example of a before/after story for the exam: "The FBX robo-dog exported to a 12 MB GLB with 4 materials and 4096 px textures. I joined meshes down to 2 materials, reduced textures to 1024 px and ran gltf-transform with Draco + WebP. The result was about 2 MB, and draw calls dropped from 8 to 2. The download got much smaller thanks to Draco/WebP; the frame rate improved because of fewer materials and smaller textures."

13 · Models in the scene, measuring, animation From lecture · SW05

Where to put the file

Vite serves everything in `public/` **as-is, at the root of the site**. `public/models/lamp.glb` → available at `/models/lamp.glb`.

Two ways to load a model in A-Frame

```
<!-- A) Directly: fine for a quick test, but the model POPS IN when it finishes loading -->
<a-entity gltf-model="url(/models/lamp.glb)"></a-entity>

<!-- B) Asset system: declared once, referenced by id. A-Frame WAITS until all
        assets are loaded before starting the scene -> no pop-in -->
<a-scene mindar-image="imageTargetSrc: /targets.mind;" ...>
  <a-assets>                                    <!-- first child of a-scene, before the camera -->
    <a-asset-item id="lamp" src="/models/lamp.glb"></a-asset-item>
  </a-assets>

  <a-camera position="0 0 0" look-controls="enabled: false"></a-camera>

  <a-entity mindar-image-target="targetIndex: 0">
    <a-entity gltf-model="#lamp" position="0 0 0" scale="0.5 0.5 0.5" rotation="90 0 0"></a-entity>
  </a-entity>
</a-scene>
```

- The image target defines the coordinate system: centre = origin, 1 unit = image width, z points toward you.
- A model with its **origin at the bottom** stands on the image.
- glTF is **+Y up**, so if the image lies flat on a table, `rotation="90 0 0"` stands the model up.
- If it **floats or sinks, the origin is wrong**: fix it in Blender.

Loading Draco-compressed models General

```
<!-- A-Frame needs to know where the Draco decoder is -->
<a-scene gltf-model="dracoDecoderPath: https://www.gstatic.com/draco/versioned/decoders/1.5.6/;">
</a-scene>
```

Measuring performance

```
<a-scene stats> ... </a-scene>     <!-- live fps, draw calls (Calls), Textures, Geometries, Vertices/Faces
-->

<!-- Eruda: developer console ON the phone (see console.log and errors) -->
<script src="https://cdn.jsdelivr.net/npm/eruda@3.4.1"></script>
<script>eruda.init()</script>
```

Alternatively, connect the phone by USB for remote debugging.

Metric (measured on the phone)	Course target (ceilings, not goals)
fps	steady **60**; **below 30 AR feels laggy**
Calls (draw calls)	under **300** for the whole scene (A-Frame best practices); **under 10 per model** (one per material and per separate mesh; a joined model with 1–3 materials = 1–3 calls)
Triangles	under **100,000** for the scene, ideally **30–50k** (Google ARCore)
Textures	as few as possible, ideally **one set of 2–3 images per model**
File size	**3–5 MB per model**, **10 MB total**

Remember: Compression changes the **download size**, not these values. **Fewer triangles, fewer materials and smaller textures** do.

Exercise: animated model

Problem: `gltf-model` loads the file, but **doesn't play the animation baked into it**. **Solution:** the `animation-mixer` component. It isn't part of A-Frame; it comes from a separate community library, **aframe-extras**. Add its script tag **after A-Frame's**.

```
<script src="https://aframe.io/releases/1.5.0/aframe.min.js"></script>
<script src="https://cdn.jsdelivr.net/gh/c-frame/aframe-extras@7.7.0/dist/aframe-extras.min.js"></script>
...
<a-assets>
  <a-asset-item id="robodog" src="/models/robodog.glb"></a-asset-item>
</a-assets>
<a-entity mindar-image-target="targetIndex: 0">
  <!-- clip: * plays all clips; or name one: clip: Walk -->
  <a-entity gltf-model="#robodog" rotation="90 0 0" scale="0.5 0.5 0.5"
            animation-mixer="clip: *; loop: repeat; timeScale: 1"></a-entity>
</a-entity>
```

Where to get models: the Ilias folder "image tracking example ressources" (Cyber Funk Studio postcard: FBX + target image), your own projects, the **Khronos glTF Sample Assets** (filter by "Animation"), or **Sketchfab** (free, downloadable, **Creative Commons**). Check that the model is rigged with an animation before downloading.

14 · Scene lighting General knowledge, no PDF

> On the calendar for SW06 as "(Scene Lighting)", in brackets, so it may have been short or skipped. These are standard A-Frame/Three.js facts.

Default lights

If you add **no** lights, A-Frame automatically adds a default **ambient + directional** light. As soon as you add **your own** light, the defaults are removed.

Light types

Type	Real-world analogy	Behaviour	Shadows
ambient	light bouncing around a room	lights everything equally, no direction; without it, shadowed sides are pitch black	no
directional	the sun	parallel rays; only the direction matters (from its position toward the target)	yes
point	a light bulb	shines in all directions from one point; gets weaker with distance	yes (expensive)
spot	a stage spotlight / torch	a cone with `angle` and `penumbra` (soft edge)	yes
hemisphere	sky above, ground below	gradient from sky colour to ground colour; cheap, natural outdoor look	no

```
<a-scene shadow="type: pcfsoft">
  <a-entity light="type: ambient; color: #FFF; intensity: 0.4"></a-entity>
  <a-entity light="type: directional; intensity: 0.8; castShadow: true" position="-1 2 1"></a-entity>
  <a-entity light="type: hemisphere; color: #CDE; groundColor: #553; intensity: 0.6"></a-entity>

  <a-entity gltf-model="#train" shadow="cast: true; receive: false"></a-entity>
  <a-plane rotation="-90 0 0" width="4" height="4" shadow="receive: true"></a-plane>
</a-scene>
```

Materials and light

- **Standard (PBR) material**: reacts to light through `metalness` and `roughness`. glTF/Principled BSDF materials are PBR.
- `shader: flat`: ignores light completely. Cheap; good for UI, stylised looks, or models with baked light.
- **Environment map / IBL** (image-based lighting): a 360° image provides lighting and reflections, so metals look realistic.

Performance and AR

- **Real-time shadows and many lights are expensive**, especially on phones. Use 1–2 lights; fake shadows with a soft "blob" texture under the object.
- **Baked lighting:** calculate light and shadow in Blender and bake it into the texture. It looks good and costs almost nothing at runtime, which makes it ideal for mobile AR.
- **AR:** virtual light should match the real room, or objects look "stuck on". 8th Wall offers **lighting estimation**; MindAR doesn't, so you choose plausible light (soft ambient + one directional from above).
- Colour: `renderer="colorManagement: true"` / `color-space="sRGB"` (as in the MindAR example) keep colours the same as in Blender.

15 · Git & GitLab General knowledge, no PDF

On the calendar for SW06. Your proposal says: "GitLab: main + feature/* branches, merge requests. Clean commit history as required."

What GitLab is

- A platform for hosting Git repositories, like GitHub. Differences: it can be **self-hosted** (universities often run their own instance), and it has **built-in CI/CD** (automatic build/test/deploy pipelines) and **GitLab Pages** (static hosting).
- Same Git underneath: clone, commit and push work exactly the same.

Vocabulary

Term	Meaning
Repository	The project folder plus its full history
Branch	A parallel line of work. `main` = stable; `feature/loader` = work in progress
Merge Request (MR)	GitLab's name for a GitHub "pull request": ask to merge a branch into main, with review and discussion
Merge conflict	Two branches changed the same lines; you decide which version to keep
Pull / fetch	Download new commits from the server
`.gitignore`	Files Git must not track: `node_modules/`, `dist/`, `.env`
CI/CD pipeline	Jobs defined in `.gitlab-ci.yml` that run automatically on every push

Feature-branch workflow (example)

```
git checkout -b feature/loading-screen      # new branch from main
# ... work ...
git add index.html src/loader.js
git commit -m "Add loading screen with progress text (F1)"
git push -u origin feature/loading-screen   # then open a Merge Request in GitLab
# after review: merge into main, delete the branch
```

Good commit messages: small commits, imperative mood, say *what and why*: "Fix train jump at loop start" rather than "stuff". Your project tags AI-generated code with `[AI]`.

Why version control matters (exam answer)

You can go back to any working version, try ideas safely on branches, see who changed what and why, collaborate without overwriting each other's work, and keep a backup on the server. A hosted repo can also **deploy** the site automatically (chapter 17).

16 · Usability features: all 8 areas, with code From lecture · SW06

"Small things with big effects" that take your experience **from a tech preview to something that feels like a finished web app**.

1. First impression

- **Page title** (`<title>`, shown in the tab) · **favicon** (small tab icon) · **link preview** (image and title when you send the link in a chat), controlled by **Open Graph** tags.

```
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1, maximum-scale=1, user-scalable=no">
  <title>Endless Line: 124 years of Swiss rail</title>
  <link rel="icon" href="/favicon.svg">
  <meta property="og:title" content="Endless Line">
  <meta property="og:description" content="Scan the ticket and travel 1902 → 2026.">
  <meta property="og:image" content="https://example.com/og-preview.jpg">
</head>
```

2. Loading screen and progress

- Show something **immediately**: logo, title, spinner, progress bar or short text.
- Show **text, not only a spinner**, so users know what they're waiting for ("Loading 3D model…").
- Remove it when the scene is ready. A-Frame fires `loaded` on `<a-scene>`. For assets, `a-asset-item` fires `progress` (loaded and total bytes), `loaded` and `error`.
- If loading takes longer than a few seconds, the user needs a sign that it's still working.

```
<div id="loader">Loading trains… <span id="pct">0</span>%</div>
<script>
  const item = document.querySelector('#train');
  item.addEventListener('progress', e => {
    const { loadedBytes, totalBytes } = e.detail;
    if (totalBytes) pct.textContent = Math.round(loadedBytes / totalBytes * 100);
  });
  item.addEventListener('error', () => loader.textContent = 'The train model could not be loaded. Please
reload.');
  document.querySelector('a-scene').addEventListener('loaded', () => loader.remove());
</script>
```

3. Start button and permissions

- **Camera and motion sensors need permission; sound needs a user gesture.**
- Optional start screen with **one clear button** ("Start AR"). **The tap is the gesture** the browser needs before it plays sound; sound that's supposed to play on load is blocked otherwise. It's also the best place to **explain why the camera is used**.
- **Handle "denied":** if the user says no, the page should say **what happened and how to fix it** (browser settings), not stay black.
- Everything needs **HTTPS**; the camera only works on a secure connection.
- **Impressum, privacy and contact:** keep a small link on every screen. A non-public course project doesn't need an Impressum. As soon as a site is **public with a commercial or professional purpose**, it usually does. In Switzerland the Impressum rule is part of the **UWG** (Unfair Competition Act) and privacy law is the **revDSG** (revised Data Protection Act). Camera images in WebAR are processed **on the device**, but if you use **analytics, forms or external services**, you must say so.

```
<button id="start">Start AR</button>
<script>
startBtn.addEventListener('click', async () => {
  try {
    await sound.play();                              // allowed now: it follows a user gesture
    const stream = await navigator.mediaDevices.getUserMedia({ video: { facingMode: 'environment' } });
    stream.getTracks().forEach(t => t.stop());       // just checking; MindAR opens the camera itself
    document.querySelector('a-scene').systems['mindar-image-system'].start();
  } catch (err) {
    if (err.name === 'NotAllowedError')
      showError('We could not access your camera. Allow it in your browser settings (aA → Website settings
→ Camera) and reload.');
    else if (err.name === 'NotFoundError')
      showError('No camera found on this device.');
    else showError('Something went wrong while starting AR. Please reload.');
  }
});
</script>
```

For a manual start like this, set `mindar-image="…; autoStart: false"`.

4. Guide the user

- **Onboarding hint:** say what to do when AR starts, in words and/or visuals: "Point your camera at the postcard". Show it while no target is found; hide it when found (`targetFound` / `targetLost`, chapter 10).
- **Tracking lost:** tell the user, and say how to get it back.
- **Show what's possible:** if something is clickable, make it **look clickable**. A short "Tap the model" hint beats a hidden interaction nobody finds.
- **Feedback for every action:** a small animation, colour change or sound when the user taps something.
- **End state** (optional): closing the tab is the normal way out, so no exit button is needed. If there's a natural next step or **call to action (CTA)**, offer it: a link to your website, "Play again".

5. When things go wrong: error messages and fallbacks

Things will fail: no camera, no WebXR, no internet, model not found. **What does the user see then?**

- **Feature detection:** before starting, check in code whether the device supports what you need (camera, WebXR, motion sensors, WebGL). **Only offer the start button if it can work**; otherwise show a message or the fallback.
- **Fallbacks, from best to simplest:** e.g. the same content in the normal browser window (touch/mouse to look around), or a still image or video with a short explanation. **Never a blank screen.**
- **Human error messages:** "We could not access your camera. Check the browser permission and reload." instead of a blank screen or a console error.
- **Test the failure paths:** turn off the camera permission, switch to airplane mode, break the model's file name, and see what the user sees.

```
function supports() {
  const secure = window.isSecureContext;                        // HTTPS?
  const camera = !!(navigator.mediaDevices && navigator.mediaDevices.getUserMedia);
  let webgl = false;
  try { const c = document.createElement('canvas');
        webgl = !!(c.getContext('webgl2') || c.getContext('webgl')); } catch (e) {}
  return { secure, camera, webgl };
}
const s = supports();
if (s.secure && s.camera && s.webgl) showStartButton();
else showFallback();     // e.g. the same 3D scene with orbit controls, or a video

window.addEventListener('offline', () => showError('You are offline. Reconnect and reload.'));
```

Remember: A fallback doesn't need to look as good as the full experience. It only needs to not be a dead end.

6. Load smarter: deferred loading and caching

- **Deferred (lazy) loading:** don't load everything at the start. Load what the first view needs, then the rest later.
- **Preload what's needed immediately**: that's what `<a-assets>` is for.
- **Caching:** the browser keeps files it has already downloaded, so the second visit is much faster. Catch: **Vite adds a hash** to the names of files it builds (`index-a1b2c3.js`), so updates are picked up automatically. **Files in `public/` keep their name**, so users may still see the **old version**. Rename or hash them when you update them (`train-v2.glb`).
- **Loading order (critical rendering path):** the page appears only as fast as the browser gets the files it needs, and order matters. **FOUC** (flash of unstyled content) = the page shows briefly without styles; **FOUT** (flash of unstyled text) = text first appears in a default font, then jumps to the web font. Fix: load CSS in the `<head>`, keep font files small, or hide the page behind the loading screen until fonts and styles are ready.

```
// lazy-load era 2 only when the user needs it
function loadEra(id, url) {
  const el = document.createElement('a-entity');
  el.setAttribute('gltf-model', `url(${url})`);
  el.addEventListener('model-loaded', () => showTrain(el));
  el.addEventListener('model-error',  () => showError('Train model missing.'));
  document.querySelector('#track').appendChild(el);
}
```

7. Comfort and readability

- **Text:** large, high contrast, short. Test on the real device.
- **Buttons:** big enough for a finger on a phone, about **44 px minimum**. Put 2D buttons and text in an **HTML overlay** rather than inside the 3D scene: a normal HTML element on top of the canvas (in WebXR sessions this is the **dom-overlay** feature).
- **Phone use:** set the **viewport meta tag**, prevent accidental scroll and zoom, and remember the **screen can turn off** while the user looks through the camera (→ Wake Lock API).

```
/* overlay above the AR canvas */
#ui { position: fixed; inset: 0; pointer-events: none; z-index: 10; }
#ui button { pointer-events: auto; min-width: 44px; min-height: 44px; font-size: 16px; }
html, body { overscroll-behavior: none; touch-action: none; }   /* no pull-to-refresh / zoom */

// keep the screen on while in AR
let lock; try { lock = await navigator.wakeLock.request('screen'); } catch (e) {}
```

8. Consistent styling

- **One colour palette and one or two fonts** for everything: page, loading screen, start button, hints, errors, end state. Write them down **once as CSS variables** so you change them in one place.
- **The same button style everywhere.** A start button that looks different from the other buttons feels like two different apps.
- **Restyle the defaults**: e.g. MindAR's built-in loading, scanning and error screens are generic. Point them to your own elements.
- **Match the 3D scene:** take your interface colours from the scene (or the other way round), so the page and the AR content feel like one piece.
- **One language, one tone, one icon style.** A custom web font must load before it shows; until then the browser shows a default font (FOUT).

```
:root { --main-color: #C8102E; --bg: #111; --text: #fff; --font: "Inter", system-ui, sans-serif; }
button { background: var(--main-color); color: var(--text); font-family: var(--font);
         border: 0; border-radius: 12px; padding: 12px 20px; min-height: 44px; }
```

The SW06 checklist

1. Something intentional is visible right after opening the page
2. Loading shows progress, or at least text
3. (Start button explains why the camera is needed)
4. Hint on how to start (what to point at)
5. Message when tracking is lost
6. Clear message when the camera is denied or features aren't supported
7. Fallback, so there's no dead end
8. Big model loaded only when needed
9. Text and buttons readable on the real device
10. Title, icon, share preview
11. One look everywhere: same colours, fonts and buttons
12. Tested on real devices and browsers, on a bad network, with permissions denied

17 · How hosting works & how to host General knowledge, no PDF

SW08: "How Hosting Works · How we will host it". Your proposal says "HTTPS static hosting (SW08)". Check with the teacher which host the course uses.

What happens when someone opens your link

```
1. User types  https://endless-line.example.ch
2. DNS         "Which IP address belongs to this domain?"  →  203.0.113.10
3. TCP + TLS   the browser connects and checks the HTTPS certificate (is this really the right server?)
4. HTTP GET    /index.html → the server sends files: HTML, then JS, CSS, .glb, .mind, audio …
5. Browser     parses HTML, runs JS → A-Frame/Three.js render with WebGL, MindAR opens the camera
               → ALL the AR logic runs on the CLIENT (the phone). The server only delivers files.
```

- **Static site:** just files (HTML, JS, CSS, models, audio), no server-side code or database. A WebAR app like yours is static, so any **static host** works: GitLab Pages, GitHub Pages, Netlify, Vercel, Cloudflare Pages, or a school web server.
- **Dynamic site:** server code (PHP, Node, database) builds pages per request, e.g. a webshop or login. Not needed here.
- **Domain + DNS:** the human-readable name points to the server's IP. **CDN** (Content Delivery Network): copies of your files on servers worldwide, so they load fast everywhere and the site stays reachable.

Dev server vs. production build

```
npm run dev       # Vite dev server: for local work only (hot reload, self-signed HTTPS)
npm run build     # creates the dist/ folder: minified, bundled, HASHED file names
npm run preview   # test the built dist/ locally before uploading
# -> upload/deploy the CONTENT of dist/ to the host
```

- Production gets a **real HTTPS certificate** from a trusted authority (e.g. **Let's Encrypt**, usually automatic on the host), so there's **no browser warning**, unlike the self-signed dev certificate.
- If the site lives in a subfolder (e.g. `username.gitlab.io/endless-line/`), set Vite's `base`, or the paths to JS and models break:

```
// vite.config.js
export default defineConfig({ base: '/endless-line/', plugins: [basicSsl()], server: { host: true } })
```

Example: automatic deploy with GitLab Pages

```
# .gitlab-ci.yml : runs on every push to main
image: node:20
pages:
  script:
    - npm ci                 # install exact versions from package-lock.json
    - npm run build          # -> dist/
    - rm -rf public && mv dist public   # GitLab Pages serves the "public" folder
  artifacts:
    paths: [public]
  rules:
    - if: $CI_COMMIT_BRANCH == $CI_DEFAULT_BRANCH
```

Netlify/Vercel do the same with settings: build command `npm run build`, publish directory `dist`.

Hosting checklist for WebAR

- HTTPS ✓ (camera!) · correct MIME types for `.glb` / `.mind` ✓ · file names lowercase, no spaces or diacritics ✓

- Test the **live URL** on iPhone + Android, on a slow network, with permissions denied ✓
- Cache: hashed build files update automatically; renamed `public/` files for updates ✓

18 · Basic security features General knowledge, no PDF

SW05 on the old calendar, SW08 on the new one. These are the standard basics for a static WebXR site.

Feature	What it protects	Example / how
HTTPS / TLS	Encrypts traffic (nobody on the Wi-Fi can read or change it) and proves the server's identity	Required for a **secure context** → camera, WebXR, sensors, Wake Lock
Permissions	The user decides about camera, microphone, motion, location	Ask only when needed, after a tap, explain why, handle "denied"
Third-party scripts / CDNs	If the CDN is hacked, its script runs on your page	Pin exact versions (`aframe@1.5.0` , not "latest"), use **SRI**, or bundle via npm
Subresource Integrity (SRI)	The browser refuses a CDN file that has been changed	`<script src="…" integrity="sha384-…" crossorigin="anonymous">`
Content Security Policy (CSP)	Limits where scripts, images and media may load from; stops injected scripts (XSS)	`<meta http-equiv="Content-Security-Policy" content="default-src 'self'; script-src 'self' https://aframe.io https://cdn.jsdelivr.net">`
No secrets in front-end code	Everything in JS can be read in DevTools	No API keys or passwords in the repo or bundle; `.env` in `.gitignore`
Dependencies	Supply-chain attacks via npm packages	`npm audit` , commit `package-lock.json` , install only known packages
Safe DOM updates	XSS through user input or URL parameters	Use `textContent` , not `innerHTML` , for untrusted text
Privacy (revDSG)	Personal data	Camera images stay on the device; analytics/forms/external services must be disclosed in a privacy note

```
<!-- Example: SRI on a CDN script -->
<script src="https://aframe.io/releases/1.5.0/aframe.min.js"
        integrity="sha384-...hash-of-the-exact-file..."
        crossorigin="anonymous"></script>

// Example: safe text output
infoTitle.textContent = trainName;      // safe
// infoTitle.innerHTML = trainName;     // dangerous if trainName comes from the URL/user
```

Remember: Security on a static WebAR site mostly means: **HTTPS**, **permissions handled respectfully**, **trusted and pinned dependencies**, **no secrets in the client**, and **being honest about data**.

19 · Your project: Endless Line From your proposal (30.10.2026)

Pitch (memorise): "Endless Line is a handheld Web AR experience about the history of Swiss rail. You scan a ticket card and a looping figure-8 track appears on it. A train runs on it endlessly and changes era as you drag through time, from 1902 to today. No app download needed."

- Type: handheld Web AR · Tracking: **image target** · Devices: iOS & Android · Team: solo.
- SBB has run since **1902** → 124 years. The figure-8 (∞) stands for the "endless" network.
- 5 eras: **1902** steam (A 3/5) · **1919** Ce 6/8 II "Krokodil" · **1964** Re 4/4 II · **1992** Re 460 · **2019** Giruno. Train, sound (steam → electric hum → modern chime) and colour mood (sepia → full colour) change with smooth transitions.
- User flow: scan the QR code on the ticket card → start (loading, camera & sound unlock) → track appears → drag the timeline → tap the train for facts.
- Unofficial student project: **no SBB logos**; SBB is only named as historical fact.

Tech stack and the "why" behind each choice

Area	Tool	Why (link to the course)
XR framework	A-Frame 1.x + **MindAR** image tracking	Covered SW02–SW04; open source (MIT); **works in Safari + Chrome with one build**, because iOS has no WebXR AR and WebXR has no image tracking
Train movement	Custom A-Frame component, parametric ∞ curve (Three.js)	Constant speed, seamless loop, the train faces along the track
Sound	Web Audio via **Howler.js**	Crossfades between eras; handles the **iOS audio unlock** (user gesture)
3D assets	Blender low-poly → GLB, **gltf-transform (Draco + WebP)**	SW05: 5 stylised trains, compressed; ≤ 3 MB total
Build & dev	**Vite**, vanilla JS (ES modules)	HTTPS dev server for phone testing; **hashed build files** (cache)
Version control	**GitLab**: main + feature/* branches, merge requests	Clean commit history
Hosting	HTTPS static hosting (SW08)	The camera needs HTTPS; weekly real-device tests

The six features

1. **F1 First impression & loading:** title, favicon, OG tags; logo within 1 s; "Loading trains… 70 %" from `a-asset-item progress`; no FOUC/FOUT; hide on `a-scene loaded`.
2. **F2 Start, permissions, errors, fallback:** feature detection (HTTPS, getUserMedia, WebGL); "Start AR" only if the checks pass; explain the camera; camera and audio only after the tap; human messages; fallback = the same scene in the browser with drag-to-orbit; failure paths tested.
3. **F3 Tracking & endless loop:** recognised within 2 s, works at 30–60 cm; constant speed; seamless loop; ≥ 30 fps on a mid-range phone; scan hint; "Ticket lost" after 1 s; MindAR UI restyled.
4. **F4 Timeline:** slider snaps to 5 eras, handle ≥ 44 px; swap with a fade/scale transition ≤ 600 ms while the train keeps moving; the year counts up; colour mood per era; optional "Play"; fast dragging never shows two trains or none.
5. **F5 Sound:** one ambient loop per era, crossfade ≤ 1 s + a whistle; only after the tap (iOS); mute button always visible; pauses when tracking is lost or the tab is hidden; ≤ 300 KB per file.
6. **F6 Info, end state, polish:** tap the train → HTML-overlay info card (≥ 16 px text, ≥ 44 px targets); "Ride again" + source link; Impressum/Privacy/"not affiliated with SBB" on every screen; Wake Lock; GLBs Draco ≤ 3 MB, first era preloaded, others deferred; CSS variables.

Stretch goals: low-poly scenery per era, DE/FR/IT/EN, pinch/rotate gestures.
AI use: Claude as a coding assistant, one feature at a time, with the acceptance criteria as "done"; commits tagged `[AI]`; no one-shotting; everything reviewed and tested on a real device. **Not AI:** concept, ticket design, train models, sound choice, final texts.
Risks → mitigation: 5 models take too long → stylised low-poly, a shared base mesh, CC models as backup · iOS audio → unlock on the start tap, test on iPhone from day one · branding → no SBB logos, "unofficial" everywhere · time → stretch goals only after F1–F6.

Example: the endless-loop component (to explain "how does the train move?")

```
AFRAME.registerComponent('follow-loop', {
  schema: { speed: { default: 0.08 } },              // loops per second (fraction of track length)
  init() {
    const pts = [];
    for (let i = 0; i < 64; i++) {                   // figure-8 = lemniscate of Gerono
      const t = (i / 64) * Math.PI * 2;
      pts.push(new THREE.Vector3(0.4 * Math.sin(t), 0, 0.4 * Math.sin(t) * Math.cos(t)));
    }
    this.curve = new THREE.CatmullRomCurve3(pts, true);  // closed = true -> seamless loop
    this.u = 0;
  },
  tick(time, dt) {
    this.u = (this.u + this.data.speed * dt / 1000) % 1;  // wraps 1 -> 0 without a jump
    const p = this.curve.getPointAt(this.u);              // "At" = by ARC LENGTH -> constant speed
    const dir = this.curve.getTangentAt(this.u);          // direction of travel
    this.el.object3D.position.copy(p);
    this.el.object3D.rotation.y = Math.atan2(dir.x, dir.z); // train faces along the track
  }
});
// <a-entity mindar-image-target="targetIndex: 0">
//   <a-entity rotation="90 0 0"> <a-entity gltf-model="#era1" follow-loop></a-entity> </a-entity>
// </a-entity>
```

Key explanation: `getPoint(t)` follows the curve's parameter, so the train would speed up and slow down. `getPointAt(u)` uses **arc length**, so equal steps of `u` are equal distances: **constant speed**. A **closed** curve plus `% 1` gives a seamless loop. Multiplying by `dt` keeps the speed the same at any frame rate.

Milestones

30.10 proposal + repo & Vite · 04.11 F1 + F2 · 18.11 F3 tracking + loop · 25.11 F4 + all trains · 02.12 F5 + F6 · 09.12 hosting & testing · 16.12 demo & oral report. **Be honest about where you are right now.**

20 · Practice questions with model answers

WebXR & standards

Q: What is WebXR, in one minute?
A: The WebXR Device API is a W3C web standard that lets a website talk to VR/AR hardware (headsets, the phone's motion sensors, controllers), the way a website talks to a mouse. The browser can show immersive VR or AR without installing an app: build once, run everywhere, no app store, no lock-in. The Immersive Web Working Group develops it; it's a Candidate Recommendation Draft and fully implemented only in Blink browsers (Chrome, Edge, Brave, Quest, PICO).

Q: Where did WebXR come from?
A: Mozilla's WebVR API proved browser VR was possible but was never ratified and is now deprecated. The W3C Immersive Web WG started in Sept 2018, the first public working draft came in Feb 2019, Chrome 79 shipped it first in Dec 2019, then Brave and Edge 81 in 2020.

Q: Why isn't WebXR a full Recommendation yet, and what would change?
A: It needs two independent engines with complete, interoperable implementations. Only Blink has one; Firefox has none and Mozilla left the WG; Safari supports only VR on Vision Pro. The charter calls it a living spec with yearly updates. As a Recommendation, WG members would commit to royalty-free patent licensing, and the W3C would recommend wide deployment.

Q: Why can't you use WebXR AR on an iPhone, and what do you use instead?
A: Safari/WebKit (and therefore every iOS browser) doesn't implement the WebXR AR module. Workarounds like 8th Wall, MindAR and AR.js skip WebXR completely and do camera access, computer vision and rendering in JavaScript/WebGL, which runs everywhere.

Tracking

Q: Explain SLAM.
A: Simultaneous Localisation And Mapping: the device tracks feature points in the camera image to build a point cloud of the room, and combines that with gyroscope and accelerometer data to know its own 6DoF position at the same time. It enables plane detection, hit test, light estimation and real-scale content. On the web it's only possible with 8th Wall, whose SLAM module is closed-source. Example: IKEA furniture placement.

Q: Marker vs. image tracking?
A: A marker is a simple black/white square with a thick border built for computers: robust, fast, many IDs, but ugly (AR.js). Image tracking recognises any known picture and calculates its position and rotation (MindAR, AR.js NFT, 8th Wall), but the image needs lots of detail and contrast.

Q: What makes a good target image, and why?
A: Varied, abundant detail, high contrast, sharp corners and lines, no repeating patterns, no large blank areas, a matte surface. The tracker needs many unique feature points; repetition makes points look alike, blank space has none, and reflections change the image.

Q: What are the limits of location-based AR?
A: GPS is only accurate to 3–10 m, the compass fails indoors or near metal, and there's no SLAM, so it only works in large open outdoor spaces. Niantic combines GPS + SLAM + VPS (built from Pokémon Go data) for centimetre accuracy.

Tools

Q: What's the difference between A-Frame and Three.js?
A: Three.js is a JavaScript library that wraps WebGL into scene, camera, lights and meshes. A-Frame is a framework on top of Three.js that describes scenes with HTML tags and adds the VR button, camera and controls automatically. Everything in A-Frame becomes Three.js underneath.

Q: Why Vite, and why HTTPS?
A: Vite is a fast local dev server that makes adding HTTPS easy (plugin-basic-ssl) and is reachable on the local network (host: true). WebXR and camera access only work in a secure context, so the phone must load the page over HTTPS. The self-signed certificate gives a warning you click through.

Q: Is it risky that Cloudflare bought Vite's company, or that MindAR is one person?
A: What matters is the license and governance. Vite stays MIT and vendor-agnostic, developed in the open. MindAR and A-Frame have few maintainers (higher risk), but the MIT license means anyone can fork and keep going.

3D models

Q: Why glTF/GLB?
A: It's the "JPEG of 3D": an open Khronos standard (ISO/IEC 12113:2022) made for efficient delivery and loading at runtime. GLB is one binary file with mesh, materials, textures and animations. +Y up, metres.

Q: What makes a model heavy, and how do you fix it?
A: Vertices (~32 B each), textures (2048^2 = 16 MB GPU), materials (one draw call each) and animations. Treat it as a budget: first merge materials and textures (atlas + bake), use 1024 px powers-of-two textures, then decimate or bake normals (under 50k triangles), and remove hidden parts and unused data.

Q: Does Draco make my scene run faster?
A: No. Draco quantizes and entropy-codes the geometry (90–95 % smaller download, decoded in a Web Worker), but the GPU renders the same triangles. FPS depends on triangles, draw calls and textures. Likewise, WebP shrinks the download but not GPU memory; only KTX2 shrinks GPU memory (4–8×).

Q: Why does my model float above the image or lie on its side?
A: In MindAR the image centre is the origin, 1 unit = image width, z points toward the camera. glTF is Y-up, so use rotation="90 0 0" to stand it up. If it floats or sinks, the origin in Blender is wrong: set it to the bottom/anchor point and apply transforms.

Q: How do you play a model's animation?
A: gltf-model doesn't play baked animations. Add aframe-extras after A-Frame and use animation-mixer="clip: *; loop: repeat".

Q: WebGL vs. WebGPU? Three.js vs. Babylon.js?
A: WebGL is the browser's connection to the GPU, with an extra layer compared to native. WebGPU (W3C, Candidate Recommendation Draft) is its successor, built for modern GPUs, giving more budget for busy scenes. Three.js is a lighter library you assemble; Babylon.js (Microsoft 2013, TypeScript) is a full engine with an inspector, physics and a node editor.

Usability, hosting, security

Q: Name usability features you'd build into a WebAR app.
A: Title, favicon, OG preview; a loading screen with text/progress; a start button that explains the camera and unlocks sound; a "point at the target" hint and a tracking-lost message; human error messages and a fallback with no dead end; lazy loading and cache-aware file names; 44 px buttons in an HTML overlay; one consistent style with CSS variables; testing failure paths.

Q: Why is sound blocked, and how do you fix it?
A: Browsers block audio until there's a user gesture. The "Start AR" tap is that gesture, so you start or unlock audio inside the click handler (Howler.js handles the iOS unlock).

Q: Users see an old model after you updated it. Why?
A: Browser caching. Vite hashes the files it builds, but files in public/ keep their name, so the browser reuses the cached old copy. Rename or hash them (train-v2.glb).

Q: How does hosting work for your app?
A: It's a static site: npm run build creates dist/, which is uploaded to a static HTTPS host (e.g. GitLab Pages via CI). DNS maps the domain to the server, TLS secures the connection with a trusted certificate, the server only delivers files, and all AR logic runs on the phone.

Q: Which basic security measures apply?
A: HTTPS (required for the camera), respectful permission handling, pinned and trusted dependencies (SRI for CDN scripts, npm audit), CSP, no secrets in front-end code, textContent instead of innerHTML, and a privacy note (camera stays on the device; revDSG).

Q: Do you need an Impressum?
A: Not for a non-public course project. As soon as a site is public with a commercial or professional purpose, usually yes (Swiss UWG). Privacy falls under the revDSG; analytics, forms or external services must be disclosed.

About your project

Q: Why MindAR and not WebXR or 8th Wall?
A: I need image tracking on a ticket card, and it has to work on iPhones. WebXR has no image tracking and no AR on Safari. MindAR is free, MIT, does image tracking in Safari and Chrome with one build, and fits A-Frame. 8th Wall's SLAM isn't needed and is closed. Risk: one maintainer, but MIT means I could fork it.

Q: How do you keep the app performant on a phone?
A: Low-poly trains with 1–3 materials and 1024 px textures, Draco + WebP via gltf-transform, ≤ 3 MB total, the first era preloaded and the others deferred, measured with stats on real phones (≥ 30 fps, ideally 60).

Q: What happens if the camera is denied or there's no internet?
A: A human message explains how to re-enable the camera in browser settings; offline or a failed model shows a message with a Reload button; and there's a fallback: the same scene in the browser, drag to orbit. Never a blank screen.

21 · Glossary & numbers to memorise

Organisations & people	
W3C	Web standards (HTML, CSS, WebXR, WebGPU)
Immersive Web WG	W3C group for WebXR (since 24.09.2018)
Khronos Group	Graphics standards (OpenGL, Vulkan, WebGL, OpenXR, glTF), since 2000
Mozilla	WebVR, Hubs (2018–2024), A-Frame's origin
Mr.doob	Ricardo Cabello, Three.js (2010)
Supermedium	Maintains A-Frame (ex-Mozilla)
Niantic Spatial	8th Wall; VPS; Pokémon Go
hiukim	Sole MindAR developer
Evan You / VoidZero	Vite; bought by Cloudflare (June 2026)
Google	Draco; ARCore guidelines
Microsoft	Babylon.js (2013)

Numbers	
2015 / 2017 / 2022	glTF 1.0 / 2.0 / ISO 12113
Dec 2015	A-Frame released
Feb 2019 · Dec 2019	First WebXR draft · Chrome 79
3–10 m	GPS accuracy
~32 B	per vertex (100k ≈ 3 MB)
16 MB	GPU memory, 2048^2 texture
90–95 %	Draco geometry reduction
4–8×	KTX2 GPU memory saving
< 50k tris	per model (course)
100k / 30–50k	ARCore triangle max / ideal
10 MB / 3–5 MB	ARCore max / course per model
1024 (2048)	texture px
60 / 30 fps	target / laggy below
< 300 / < 10	draw calls scene / model
44 px	min touch target

Term	Short definition
Secure context	Page loaded over HTTPS (or localhost); needed for camera, WebXR, sensors
6DoF	Rotation + position tracking (3DoF = rotation only)
Feature points / point cloud	Distinctive image spots tracked by SLAM / the 3D map they form
Hit test	Tap on 2D screen → find the real surface in 3D to place content
Anchor	A fixed, persistent point in the real world that content sticks to
Draw call	One command to the GPU to draw a mesh/material: fewer = faster
Quantization / entropy coding	Storing numbers with fewer bits / packing frequent values more tightly (Draco)
Principled BSDF	Blender's PBR shader, the only one that exports to glTF
Normal map	Texture that fakes fine surface detail without extra geometry
Baking	Pre-computing colour, light or detail into a texture
Web Worker	Background JS thread, so heavy work (Draco decode) doesn't freeze the page
dom-overlay	WebXR feature for showing HTML on top of the immersive AR view
Open Graph (og:)	Meta tags that control link previews in chats and social media
FOUC / FOUT	Flash of unstyled content / unstyled text
Lazy (deferred) loading	Load assets only when they're needed
Hashed file names	Vite's `app-3f9a1c.js` : a new name per version → cache never serves a stale copy
Feature detection	Check in code if the browser supports something before using it
Fallback	A simpler version when the full experience can't run, never a dead end
CTA	Call to action, e.g. "Ride again", a link to more info
UWG / revDSG	Swiss Unfair Competition Act (Impressum) / revised Data Protection Act
MIT license	Permissive open-source license: anyone may use, modify and fork
Static site / CDN	Just files, no server code / worldwide network of file copies
SRI / CSP	Integrity hash for external files / policy limiting where resources may load from

Good luck tomorrow! Structure for every answer: definition → example → link to Endless Line.